

Synthèse – Projet Firmware UEFI Microkernel basé sur coreboot

1. Objectif du Projet

Développer un firmware UEFI compliant from scratch, sans utiliser EDK2, avec une architecture microkernel claire, modulaire et extensible. Objectifs principaux : - Simplifier le build system - Réduire la complexité structurelle - Comprendre profondément l'architecture UEFI - Créer une alternative moderne et maintenable

2. Plateforme Cible Initiale

Base matérielle : coreboot Environnement : QEMU x86_64 (plateforme moderne, PCIe, ACPI, APIC, NVMe possible) Stratégie : laisser coreboot gérer l'initialisation bas niveau (RAM, CPU, PCI), et implémenter un payload UEFI personnalisé.

3. Architecture Microkernel Firmware

Le firmware est structuré comme un microkernel avec modules séparés : - fw_core : orchestration générale et EFI_SYSTEM_TABLE - memory : gestion mémoire, AllocatePool, GetMemoryMap - handle_db : base de données des handles - protocol : interfaces UEFI structurées proprement - dispatcher : initialisation des drivers - image_loader : chargeur PE/COFF - runtime : Runtime Services - drivers : PCI, NVMe, Console, GOP - setup : moteur de configuration firmware - platform/ : couches spécifiques plateforme

4. Moteur Setup (Menu Firmware)

Un moteur de menu intégré permet la configuration matérielle : - Menu texte initial basé sur Simple Text Output/Input - Modèle déclaratif simple (forms en C pur) - Liaison directe avec Variable Services (Get/SetVariable) - Support BootOrder modifiable - Possibilité d'extensions par plateforme
Objectif : éviter la complexité HII/IFR tout en restant conforme à la spec.

5. Plan de Développement

- Phase 0 : Build + payload minimal (Hello firmware)
- Phase 1 : Memory init + allocator (Boot Services base)
- Phase 2 : EFI_SYSTEM_TABLE minimal + console
- Phase 3 : Handle database + modèle protocole
- Phase 4 : Console complète + PCI enumeration

- Phase 5 : PE/COFF loader + EFI_LOADED_IMAGE_PROTOCOL
- Phase 6 : Boot Services complets + GetMemoryMap + ExitBootServices
- Phase 7 : ACPI minimal (RSDP, FADT, MADT)
- Phase 8 : NVMe driver minimal + File System Protocol
- Phase 9 : Boot Manager minimal (BOOTX64.EFI)
- Phase 10 : Variable store (volatile puis persistant)
- Phase 11 : Setup engine texte + BootOrder modifiable
- Phase 12 : SMP bringup (multi-core)
- Phase 13 : Extensions futures (Secure Boot, Capsule, SMM, TPM, réseau)

6. Vision Finale

Résultat attendu : - Firmware UEFI compliant - Architecture claire et modulaire - Build simple (Makefile/CMake minimal) - Code compréhensible (<100k LOC cible) - Extensible multi-plateformes
- Base solide pour ajouts sécurité et fonctionnalités avancées